แนวคิดหลักและคำอธิบาย

HftBacktest

เฟรมเวิร์กทดสอบย้อนหลังสำหรับ การซื้อขายความถี่สูงและ Market Making

https://hftbacktest.readthedocs.io/en/latest/
https://github.com/nkaz001/hftbacktest

เกี่ยวกับเอกสารนี้

เอกสารนี้อธิบาย**แนวคิดหลัก**ของ HftBacktest เป็นภาษาไทยด้วยถ้อยคำของผู้จัดทำ โดยอ้างอิงจากเอกสารทางการและซอร์สโค้ดของโครงการ

HftBacktest เขียนด้วย **Rust** และใช้งานผ่าน **Python (Numba)** ไม่มี API สำหรับ C/C++ เอกสารนี้จึง**ไม่เน้นวิธีเรียกใช้ API** แต่เน้นว่า “ระบบทดสอบย้อนหลังที่สมจริงต้องมีอะไรบ้าง และทำงานอย่างไร” เพื่อให้นำไปออกแบบระบบของตัวเองด้วย C++/CUDA ได้

ท้ายเอกสารมีโค้ด C++/CUDA ที่ผู้จัดทำเขียนขึ้นเอง ตามแนวคิดแบบจำลองตำแหน่งในคิวของ HftBacktest และ**คอมไพล์และรันจริงแล้ว**บน RTX 5060 Ti (CUDA 13.4)

Contents

1 HftBacktest คืออะไร **2**

2 รูปแบบข้อมูล: เหตุการณ์ (Event) **3**

3 สถาปัตยกรรม: สองมุมมองของเวลา **4**

4 แบบจำลองความหน่วง (Latency models) **4**

5 แบบจำลองตำแหน่งในคิว (Queue position models) **5**

6 แบบจำลองตลาด (Exchange models) **7**

7 รูปแบบการเขียนกลยุทธ์และค่าที่ต้องตั้ง **7**

8 นำแนวคิดไปเขียนใน C++/CUDA **8**

8.1 โค้ด: ระเบียนเหตุการณ์และแบบจำลองคิว 8

8.2 ทดลอง: เล่นซ้ำสถานการณ์เล็ก ๆ และกวาดค่าพารามิเตอร์บน GPU 10

8.3 การแบ่งงานบน GPU: อะไรทำขนานได้ 11

8.4 ขั้นตอนที่แนะนำสำหรับโครงงาน 11

9 ข้อสมมุติ ข้อจำกัด และหลักการตรวจสอบ **12**

10 อภิธานศัพท์ **12**

แหล่งอ้างอิง **12**

1 HftBacktest คืออะไร

สาระจากแหล่งอ้างอิง (สรุปความ) — หน้าแรกของเอกสารและ README บน GitHub

- เป็นเฟรมเวิร์กโอเพนซอร์ส (MIT License) สำหรับพัฒนาและทดสอบกลยุทธ์การซื้อขายความถี่สูง (HFT) และ market making
- จุดเด่นคือการจำลองให้สมจริงในสามเรื่อง: **ความหน่วงของข้อมูลตลาด (feed latency)**, **ความหน่วงของคำสั่ง (order latency)** และ**ตำแหน่งในคิว (queue position)** ของคำสั่งที่รออยู่
- เล่นซ้ำ (replay) ข้อมูลสมุดคำสั่งซื้อขายและข้อมูลการซื้อขายระดับ tick ทุกเหตุการณ์ และสร้างสมุดคำสั่งซื้อขายขึ้นใหม่จากข้อมูลระดับ L2 (Market-By-Price) และ L3 (Market-By-Order)
- รองรับหลายสินทรัพย์และหลายตลาดพร้อมกัน, แบบจำลองความหน่วงและตำแหน่งในคิวทั้งแบบสำเร็จรูปและแบบกำหนดเอง
- แกนหลักเขียนด้วย Rust (มีบน crates.io), ใช้งานจาก Python โดยให้กลยุทธ์รันในฟังก์ชันที่คอมไพล์ด้วย Numba และมีบอทซื้อขายจริงสำหรับ Binance Futures และ Bybit (เฉพาะ Rust) ที่ใช้โค้ดกลยุทธ์ชุดเดียวกับการทดสอบย้อนหลัง
- ปรัชญาของโครงการ: เน้น**ความแม่นยำ**มากกว่าการมองโลกในแง่ร้าย ผลทดสอบไม่ควรซ่อนข้อได้เปรียบเล็ก ๆ ที่มีจริง และไม่ควรขยายเกินจริง ขั้นตอนที่แนะนำคือ ตรวจก่อนว่าผลทดสอบย้อนหลังตรงกับผลซื้อขายจริงในอดีต แล้วจึงค่อยปรับแต่งกลยุทธ์

อธิบายแบบง่าย — ทำไมการทดสอบย้อนหลัง HFT จึงยากกว่าปกติ

การทดสอบย้อนหลังกลยุทธ์ทั่วไป (เช่น ซื้อขายรายวัน) มักสมมุติว่า “ส่งคำสั่งแล้วได้ราคาปิดทันที” แต่ในระดับไมโครวินาที สมมุติฐานนั้นผิดอย่างมาก

1. **ข้อมูลมาถึงช้ากว่าเหตุการณ์จริง:** สิ่งที่เราเห็นคืออดีตเมื่อไม่กี่มิลลิวินาทีก่อน
2. **คำสั่งไปถึงตลาดช้า:** ระหว่างทาง ราคาอาจเปลี่ยนไปแล้ว
3. **คำสั่งแบบจำกัดราคาต้องเข้าคิว:** วางที่ราคาเดียวกับคนอื่น ไม่ได้แปลว่าจะได้ซื้อขาย ต้องรอคนที่อยู่ข้างหน้าในคิวหมดก่อน

ถ้าไม่จำลองสามเรื่องนี้ ผลทดสอบของกลยุทธ์ market making จะ**ดีเกินจริง**มาก เพราะคิดว่าได้ซื้อขายทุกครั้งที่ราคาแตะ HftBacktest จึงสร้างมาเพื่อแก้ปัญหานี้โดยตรง

สรุปสั้น ๆ — ใช้กับ C/C++/CUDA ได้อย่างไร

- HftBacktest ไม่มี API ภาษา C++ แต่**แนวคิดและแบบจำลองทั้งหมดนำมาเขียนใหม่ใน C++/CUDA ได้** (หัวข้อ 8 มีตัวอย่าง)
- ใช้ HftBacktest (Python/Rust) เป็น**ตัวอ้างอิง (reference)** สำหรับตรวจว่าเครื่องมือ C++/CUDA ที่เขียนเองให้ผลถูกต้อง
- รูปแบบข้อมูลของ HftBacktest (หัวข้อ 2) เป็นโครงสร้างขนาดคงที่ อ่าน/เขียนจาก C++ ได้ง่าย

2 รูปแบบข้อมูล: เหตุการณ์ (Event)

สาระจากแหล่งอ้างอิง (สรุปความ) — หน้า Data ของเอกสาร

ข้อมูลตลาดถูกเก็บเป็นอาร์เรย์ของระเบียนที่มีโครงสร้างคงที่ แต่ละระเบียนแทน**หนึ่งเหตุการณ์** ประกอบด้วย 8 ฟิลด์

ฟิลด์	ชนิด	ความหมาย
`ev`	u64	ชุดบิตแฟล็กบอกชนิดเหตุการณ์ ฝั่งซื้อ/ขาย และกฎการเรียงลำดับ
`exch_ts`	i64	เวลาที่เหตุการณ์เกิดขึ้นที่ตลาด
`local_ts`	i64	เวลาที่เครื่องของเราได้รับเหตุการณ์
`px`	f64	ราคา
`qty`	f64	ปริมาณ
`order_id`	u64	หมายเลขคำสั่ง (ใช้เฉพาะข้อมูล L3)
`ival`	i64	สำรองไว้ (จำนวนเต็ม)
`fval`	f64	สำรองไว้ (ทศนิยม)

แฟล็ก หลัก ได้แก่ ฝั่ง ซื้อ/ขาย (BUY_EVENT/SELL_EVENT), การ เปลี่ยนแปลง สมุด คำ สั่ง/การ ซื้อ ขาย (DEPTH_EVENT/TRADE_EVENT) และกฎการเรียงลำดับ (EXCH_EVENT/LOCAL_EVENT)

กฎสำคัญ: เหตุการณ์ที่มีแฟล็ก EXCH_EVENT ต้องเรียงตาม `exch_ts` และเหตุการณ์ที่มีแฟล็ก LOCAL_EVENT ต้องเรียงตาม `local_ts` ทั้งสองลำดับอาจไม่ตรงกัน หนึ่งเหตุการณ์จึงอาจมีทั้งสองแฟล็ก และ `local_ts` ต้องมากกว่า `exch_ts` เสมอ (ความหน่วงต้องเป็นบวก) ถ้าข้อมูลมีความหน่วงติดลบ (มักเกิดจากนาฬิกาไม่ตรงกัน) ให้แก้โดยซิงก์เวลาให้ดีขึ้น (เช่น PTP) หรือบวกค่าคงที่ชดเชย ไลบรารีมีฟังก์ชันช่วยเรียงลำดับและตรวจลำดับให้

อธิบายแบบง่าย — ทำไมต้องมีสองเวลา

- `exch_ts` = “ความจริง” ที่ตลาด ใช้จำลองฝั่งตลาด (การจับคู่คำสั่ง)
- `local_ts` = “สิ่งที่กลยุทธ์รู้” ใช้จำลองฝั่งเรา (กลยุทธ์ตัดสินใจจากข้อมูลที่มาถึงแล้วเท่านั้น)
- **feed latency** = `local_ts` − `exch_ts`

การเก็บสองเวลาทำให้ระบบจำลองได้ว่า “ตลาดเปลี่ยนไปแล้ว แต่กลยุทธ์ยังไม่รู้” ซึ่งเป็นแหล่งของความเสียเปรียบหลักใน HFT

อธิบายแบบง่าย — ข้อมูล L2 กับ L3

- **L2 / Market-By-Price (MBP):** บอกเพียง “ราคา p มีปริมาณรวม q” ไม่รู้ว่ามีกี่คำสั่ง หรือใครอยู่ก่อนหลัง ตำแหน่งในคิวต้อง**ประมาณ** (หัวข้อ 5)
- **L3 / Market-By-Order (MBO):** บอกทุกคำสั่งแยกกันด้วย `order_id` จึงติดตามลำดับในคิวได้ตรงกว่า

ตัวอย่าง — ระเบียนข้อมูลใน C++ ที่มีโครงสร้างเดียวกัน

ระเบียนข้างต้นมี 8 ฟิลด์ ฟิลด์ละ 8 ไบต์ รวม **64 ไบต์** เท่ากับขนาด cache line ของ CPU ส่วนใหญ่ เขียนเป็น `struct` ใน C++ ได้ตรง ๆ (ดูไฟล์ `hft_core.hpp` ในหัวข้อ 8) ข้อมูลหนึ่งวันของสินทรัพย์ที่ซื้อขายคึกคักอาจมีหลายสิบล้านเหตุการณ์ คือหลาย GB จึงควรอ่านแบบ memory-map หรือแบ่งเป็นช่วง

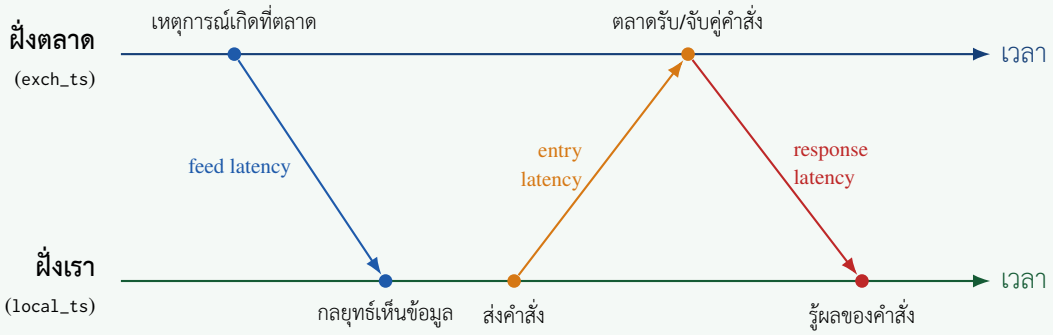
ข้อควรระวัง

ค่าตัวเลขของแฟล็กใน `hft_core.hpp` ของคู่มือนี้**เป็นค่าที่ผู้จัดทำกำหนดเอง** ถ้าจะอ่านไฟล์ข้อมูลของ HftBacktest โดยตรง ต้องใช้ค่าคงที่ตามหน้า Constants ในเอกสารอ้างอิง API ของ HftBacktest

3 สถาปัตยกรรม: สองมุมมองของเวลา

อธิบายแบบง่าย

ระบบทดสอบย้อนหลังแบบนี้แยกโลกออกเป็นสองฝั่งที่สื่อสารกันผ่านความหน่วง



- **ฝั่งตลาด** เล่นซ้ำเหตุการณ์ตาม `exch_ts` ดูแลสมุดคำสั่งซื้อขาย คิว และการจับคู่คำสั่งของเรา
- **ฝั่งเรา** เล่นซ้ำเหตุการณ์ตาม `local_ts` กลยุทธ์มองเห็นได้เฉพาะสิ่งที่มาถึงฝั่งนี้แล้ว
- ทุกการเดินทางข้ามฝั่งมีความหน่วง ผู้ทดสอบจึงต้องเดินเวลาของทั้งสองฝั่งไปพร้อมกันโดยประมวลผลเหตุการณ์ที่เวลาน้อยที่สุดก่อนเสมอ (event-driven) เพื่อไม่ให้ฝั่งเรา “รู้อนาคต”

4 แบบจำลองความหน่วง (Latency models)

สาระจากแหล่งอ้างอิง (สรุปความ) — หน้า Latency Models ของเอกสาร

- **Feed latency:** ความหน่วงตั้งแต่ตลาดเผยแพร่เหตุการณ์จนเรารับได้ วัดจากสองเวลาในข้อมูล
- **Order entry latency:** ความหน่วงตั้งแต่เราส่งคำสั่งจนระบบจับคู่ของตลาดประมวลผล
- **Order response latency:** ความหน่วงตั้งแต่ตลาดประมวลผลจนเราได้รับผลตอบกลับ (รวมถึงการแจ้งว่าได้ซื้อขายแล้ว)
- แบบจำลองสำเร็จรูป:
  - **ConstantLatency:** กำหนดค่าคงที่เองทั้งขาไปและขากลับ ง่ายที่สุด
  - **IntpOrderLatency:** ประมาณค่าโดยการ interpolate จากข้อมูลความหน่วงที่บันทึกจริง ได้จากการส่ง “คำสั่งที่ไม่มีทางถูกจับคู่” เป็นระยะเพื่อวัดเวลาไป–กลับโดยไม่กระทบตลาด แม่นที่สุดถ้าข้อมูลละเอียดพอ
  - **FeedLatency:** เมื่อไม่มีข้อมูลความหน่วงของคำสั่งจริง สร้างความหน่วงของคำสั่งเทียมจาก feed latency
  - **กำหนดเอง:** เขียนตาม trait `LatencyModel` ใน Rust

ตัวอย่าง — ความหน่วงทำให้ผลเปลี่ยนอย่างไร

สมมุติ feed latency 2 ms, entry latency 3 ms, response latency 3 ms

- เวลา 0 ms: ที่ตลาด ราคาเสนอขายดีที่สุดเปลี่ยนจาก 100.10 เป็น 100.05
- เวลา 2 ms: กลยุทธ์เพิ่งเห็นการเปลี่ยนแปลง ตัดสินใจยกเลิกคำสั่งขายที่ 100.10 แล้ววางใหม่
- เวลา 5 ms: คำสั่งยกเลิกเพิ่งไปถึงตลาด ถ้าระหว่าง 0–5 ms มีคนมาซื้อที่ 100.10 คำสั่งเดิมของเราอาจถูกจับคู่ไปแล้ว
- เวลา 8 ms: กลยุทธ์เพิ่งรู้ว่ายกเลิกสำเร็จหรือไม่

รวมแล้วกลยุทธ์ “ตามหลังตลาด” ประมาณ 8 ms ในการตอบสนองหนึ่งรอบ ระบบทดสอบที่ไม่จำลองเรื่องนี้จะให้ผลดีเกินจริง

5 แบบจำลองตำแหน่งในคิว (Queue position models)

อธิบายแบบง่าย — ปัญหา

ที่ราคาหนึ่ง คำสั่งแบบจำกัดราคาเข้าคิวตามลำดับเวลา (price–time priority) เราจะได้ซื้อขายเมื่อคนที่อยู่**ข้างหน้า**เราหมดก่อน แต่ข้อมูล L2 บอกแค่ปริมาณรวมของระดับราคา เมื่อปริมาณลดลงเพราะมีคนยกเลิก เรา**ไม่รู้ว่าคนที่ยกเลิกอยู่ข้างหน้าหรือข้างหลังเรา** จึงต้องใช้แบบจำลองประมาณ

สัญลักษณ์: **front** = ปริมาณข้างหน้าเรา, **back** = ปริมาณข้างหลังเรา (ข้อมูลตลาดที่เล่นซ้ำไม่รวมคำสั่งของเราเอง)

สาระจากแหล่งอ้างอิง (สรุปความ) — หน้า Order Fill ของเอกสาร และซอร์สโค้ดของแบบจำลองคิว

- เมื่อเราวางคำสั่ง: front = ปริมาณที่มีอยู่แล้วที่ราคานั้น (เราต่อท้ายคิว)
- เมื่อมีการซื้อขายที่ราคาของเรา ปริมาณ x: front ลดลง x

- **RiskAverseQueueModel (ระวังที่สุด):** ถือว่าการยกเลิกเกิด**ข้างหลัง**เราเสมอ ตำแหน่งเราจะเลื่อนเฉพาะเมื่อมีการซื้อขาย (และ front ต้องไม่เกินปริมาณรวมของระดับราคา: front $\leftarrow \min(\text{front},\ \text{ปริมาณใหม่})$)
- **ProbQueueModel:** แบ่งปริมาณที่ลดลง (ที่ไม่ได้เกิดจากการซื้อขาย) ระหว่างข้างหน้าและข้างหลังด้วยความน่าจะเป็น ตำแหน่งจึงเลื่อนได้แม้ไม่มีการซื้อขาย
- ฟังก์ชันความน่าจะเป็น f ต้องมี $f(0) = 0$ และ $f(1) = 1$ ในรูปตำแหน่งสัมพัทธ์ มีแบบกำลัง (power) และแบบลอการิทึม (log)
- **เงื่อนไขการได้ซื้อขาย:** เมื่อ front ติดลบเกินครึ่งหนึ่ง lot (ปัดเศษ $-\text{front}/\text{lot}$ แล้วได้มากกว่า 0) ถือว่าได้ซื้อขาย แล้วตั้ง front เป็น 0
- ข้อสมมุติ: คำสั่งของเราเล็กพอที่จะไม่กระทบตลาด, ประมาณตำแหน่งในคิวจากข้อมูล MBP ได้ และควรตรวจผลกับการซื้อขายจริงแล้วปรับแบบจำลอง

อ่านโค้ดทีละส่วน — สูตรของ ProbQueueModel

เมื่อปริมาณของระดับราคาเปลี่ยนจาก prev เป็น new

1. ส่วนที่ลดลงโดยไม่ได้มาจากการซื้อขาย: $\Delta = (\text{prev} - \text{new}) - (\text{ปริมาณซื้อขายสะสมตั้งแต่อัปเดตครั้งก่อน})$
2. ถ้า $\Delta < 0$ (ระดับราคาโตขึ้น): front $\leftarrow \min(\text{front}, \text{new})$ แล้วจบ
3. back = prev − front และ $p = P(\text{การลดเกิดข้างหลังเรา})$
4. front $\leftarrow \min\Big(\text{front} - (1-p)\Delta + \min(\text{back} - p\Delta,\ 0),\ \text{new}\Big)$

ความหมาย: สัดส่วน $(1-p)$ ของการยกเลิกเกิดข้างหน้าเรา ถ้าข้างหลังมีไม่พอให้ลด ส่วนที่เกินก็ต้องมาจากข้างหน้า

ฟังก์ชันความน่าจะเป็นที่มีให้เลือก (ตามซอร์สโค้ด)

ชื่อ	p (การลดเกิดข้างหลัง)
PowerProbQueueFunc ($f(x) = x^n$)	$\dfrac{\text{back}^n}{\text{back}^n + \text{front}^n}$
PowerProbQueueFunc2	$\dfrac{\text{back}^n}{(\text{back} + \text{front})^n}$
PowerProbQueueFunc3	$1 - \left(\dfrac{\text{front}}{\text{front} + \text{back}}\right)^n$
LogProbQueueFunc ($f(x) = \ln(1+x)$)	$\dfrac{\ln(1+\text{back})}{\ln(1+\text{back}) + \ln(1+\text{front})}$
LogProbQueueFunc2	$\dfrac{\ln(1+\text{back})}{\ln(1+\text{back}+\text{front})}$

ตัวอย่าง — คำนวณด้วยมือ: เข้าคิวตอนมี 10 lot, ต่อมามีคนต่อท้าย 4 lot แล้วมีการยกเลิก 4 lot

ก่อนยกเลิก: front = 10, back = 4, ปริมาณลด $\Delta = 4$

แบบจำลอง	p	front ใหม่	ความหมาย
Risk-averse	–	$\min(10, 10) = 10$	ถือว่าคนข้างหลังยกเลิกทั้งหมด
Power $n = 1$	$\frac{4}{4+10} = 0.286$	$10 - 0.714 \times 4 = 7.143$	
Power $n = 2$	$\frac{16}{16+100} = 0.138$	$10 - 0.862 \times 4 = 6.552$	
Power $n = 3$	$\frac{64}{64+1000} = 0.060$	$10 - 0.940 \times 4 = 6.241$	ยิ่ง n ใหญ่ ยิ่งเชื่อว่าคนข้างหน้ายกเลิก

เหตุผลของ n: เมื่อ n ใหญ่ ฝั่งที่มีปริมาณมากกว่า (ในที่นี้คือข้างหน้า 10 lot) จะได้รับส่วนของการยกเลิกมากขึ้น ตัวเลขทั้งหมดในตารางนี้ตรงกับผลที่ได้จากโค้ดในหัวข้อ 8

6 แบบจำลองตลาด (Exchange models)

สาระจากแหล่งอ้างอิง (สรุปความ) — หน้า Order Fill ของเอกสาร

- **NoPartialFillExchange (ค่าเริ่มต้น):** คำสั่งได้ซื้อขายทั้งหมดหรือไม่ได้เลย คำสั่งซื้อได้ซื้อขายเมื่อ (1) ราคาของเราไปถึงราคาเสนอขายดีที่สุด หรือ (2) มีการซื้อขายฝั่งขายที่ราคาต่ำกว่าราคาของเรา หรือ (3) เราอยู่หัวคิวและมีการซื้อขายที่ราคาของเราพอดี (คำสั่งขายกลับด้านกัน)
- **PartialFillExchange:** เงื่อนไขเหมือนกัน แต่กรณีที่ได้ซื้อขายเพราะอยู่หัวคิว ปริมาณที่ได้จะจำกัดด้วยปริมาณที่เหลือของการซื้อขายนั้น จึงได้ซื้อขายบางส่วนได้
- **คำสั่งที่กินสภาพคล่อง (liquidity-taking):** แบบแรกถือว่าได้ทั้งหมดที่ราคาดีที่สุดไม่ว่าขนาดเท่าไร แบบที่สองจับคู่ตามปริมาณในสมุด แต่ทั้งสองแบบ**ไม่จำลองปฏิกิริยาของตลาด**ต่อคำสั่งขนาดใหญ่ ผลจึงไม่สมจริงถ้าคำสั่งใหญ่

ข้อควรระวัง — ข้อจำกัดพื้นฐานของการเล่นซ้ำข้อมูล

ข้อมูลที่เล่นซ้ำคืออดีตที่**ไม่มีเราอยู่ในนั้น** ตลาดในข้อมูลจึงไม่ตอบสนองต่อคำสั่งของเรา ถ้ากลยุทธ์ใช้คำสั่งขนาดใหญ่หรือซื้อขายบ่อยจนกระทบราคา ผลทดสอบจะดีเกินจริง ต้องตรวจกับการซื้อขายจริงเสมอ

7 รูปแบบการเขียนกลยุทธ์และค่าที่ต้องตั้ง

สาระจากแหล่งอ้างอิง (สรุปความ) — หน้า Getting Started ของเอกสาร

ตั้งค่าสินทรัพย์แต่ละตัว: ไฟล์ข้อมูล (และ snapshot เริ่มต้นถ้ามี), แบบจำลองความหน่วง, แบบจำลองคิว, แบบจำลองตลาด, ค่าธรรมเนียม maker/taker (ค่าติดลบคือได้เงินคืน หรือ rebate), ขนาด tick, ขนาด lot และชนิดสินทรัพย์ (linear หรือ inverse)

วงจรหลักของกลยุทธ์:

1. เดินเวลาไปข้างหน้าด้วย `elapse(duration)` หรือรอจนมีข้อมูลใหม่/ผลตอบกลับของคำสั่ง (`wait_next_feed`, `wait_order_response`) จนกว่าข้อมูลจะหมด
2. อ่านสถานะ: สมุดคำสั่งซื้อขาย (`depth`) ราคาเสนอซื้อ/ขายดีที่สุด, สถานะการถือ (`position`), เงินคงเหลือและค่าธรรมเนียม, คำสั่งที่เปิดอยู่ (`orders`)
3. ตัดสินใจ: ส่งคำสั่งซื้อ/ขาย (ระบุหมายเลขคำสั่ง ราคา ปริมาณ อายุคำสั่งเช่น GTC หรือ GTX ซึ่งคือ post-only และชนิด LIMIT/MARKET), ยกเลิกคำสั่ง, ล้างคำสั่งที่ไม่ active แล้ว
4. บันทึกสถานะด้วย Recorder เพื่อนำไปคำนวณสถิติและวาดกราฟ

คำสั่งมีสถานะ เช่น NEW, FILLED, CANCELED, EXPIRED และคำขอ (ส่งใหม่/ยกเลิก) จะค้างอยู่จนกว่าความหน่วงขาตอบกลับจะผ่านไป

อธิบายแบบง่าย — หน้าตาของวงจรกลยุทธ์ market making (ผู้จัดทำเขียนเป็น pseudo-code)

```
while backtester.advance_time(100 ms) == OK:
    bid, ask = best prices from the depth seen locally
    q = current position
    r = reservation price (e.g. Avellaneda-Stoikov: mid - q*gamma*sigma^2*(T-t))
    half = half spread
    cancel resting orders that are too far from (r - half, r + half)
    submit post-only BUY at round_down_to_tick(r - half) if not already there
    submit post-only SELL at round_up_to_tick(r + half) if not already there
    record position, cash, mid-price
```

สังเกตว่ากลยุทธ์จาก Avellaneda & Stoikov (2008) ใส่ลงในวงจรนี้ได้ทันที แต่ต่างจากการจำลองในบทความเดิมตรงที่ ราคาต้องปัดเป็น tick, มีความหน่วง, ต้องเข้าคิว และมีค่าธรรมเนียม

8 นำแนวคิดไปเขียนใน C++/CUDA

8.1 โค้ด: ระเบียนเหตุการณ์และแบบจำลองคิว

ไฟล์ด้านล่างผู้จัดทำเขียนขึ้นเองตามแนวคิดในหัวข้อ 2 และ 5 ใช้ได้ทั้งบน CPU และ GPU (ฟังก์ชันประกาศเป็น `__host__ __device__` เมื่อคอมไพล์ด้วย nvcc)

```cpp
// hft_core.hpp : minimal C++/CUDA sketch of HftBacktest-style concepts
// (event record, queue-position models, fill check).
// Written for teaching; it follows the ideas described in the HftBacktest
// documentation but is NOT a port of its source code, and the flag values
// below are this sketch's own choice (not HftBacktest's numeric constants).
#pragma once
#include <cstdint>
#include <cmath>

#ifdef __CUDACC__
#define HD __host__ __device__
#else
#define HD
#endif

namespace hft {

// ---- event flags (own values for this sketch) ----
enum : uint64_t {
    BUY_EVENT = 1ull << 0,
    SELL_EVENT = 1ull << 1,
    DEPTH_EVENT = 1ull << 2,
    TRADE_EVENT = 1ull << 3,
    EXCH_EVENT = 1ull << 4, // must be ordered by exch_ts (exchange side sees it)
    LOCAL_EVENT = 1ull << 5 // must be ordered by local_ts (strategy sees it)
};

// ---- one market-data record: 8 fields x 8 bytes = 64 bytes ----
```

```cpp
struct Event {
    uint64_t ev; // bit flags
    int64_t exch_ts; // time the event happened at the exchange (ns)
    int64_t local_ts; // time the event reached our machine (ns)
    double px; // price
    double qty; // quantity (new level size for depth, size for trades)
    uint64_t order_id; // only used by L3 (market-by-order) data
    int64_t ival; // reserved
    double fval; // reserved
};
static_assert(sizeof(Event) == 64, "Event should be 64 bytes");

HD inline int64_t feed_latency(const Event& e) { return e.local_ts - e.exch_ts; }

// ---- queue position of one resting order at its price level ----
struct QueuePos {
    double front; // estimated quantity ahead of our order
    double cum_trade; // traded quantity since the last depth update
};

// p = probability that a size decrease happened BEHIND our order
struct PowerProb { // f(x) = x^n
    double n;
    HD double operator()(double front, double back) const {
        double fb = pow(back, n), ff = pow(front, n);
        return (fb + ff) > 0.0 ? fb / (fb + ff) : 1.0; // guard 0/0 (own choice)
    }
};
struct LogProb { // f(x) = ln(1 + x)
    HD double operator()(double front, double back) const {
        double lb = log1p(back), lf = log1p(front);
        return (lb + lf) > 0.0 ? lb / (lb + lf) : 1.0;
    }
};

// our order joins the back of the queue
HD inline void on_new_order(QueuePos& q, double level_qty) {
    q.front = level_qty;
    q.cum_trade = 0.0;
}

// a trade happened exactly at our price
HD inline void on_trade(QueuePos& q, double trade_qty) {
    q.front -= trade_qty;
    q.cum_trade += trade_qty;
}

// risk-averse model: cancels are assumed to be behind us,
// the queue ahead can only shrink when the whole level is smaller than it
HD inline void on_depth_risk_averse(QueuePos& q, double new_qty) {
    q.front = fmin(q.front, new_qty);
    q.cum_trade = 0.0;
}
```

```
// probabilistic model: split a (non-trade) decrease between front and back
template <class Prob>
HD inline void on_depth_prob(QueuePos& q, double prev_qty, double new_qty, Prob prob) {
    double chg = (prev_qty - new_qty) - q.cum_trade; // decrease not explained by trades
    q.cum_trade = 0.0;
    if (chg < 0.0) { // level grew: nothing ahead of us left
        q.front = fmin(q.front, new_qty);
        return;
    }
    double back = prev_qty - q.front;
    double p = prob(q.front, back);
    double est = q.front - (1.0 - p) * chg + fmin(back - p * chg, 0.0);
    q.front = fmin(est, new_qty);
}

// filled quantity once the queue ahead is used up (rounded to lots)
HD inline double take_fill(QueuePos& q, double lot) {
    double exec = round(-q.front / lot);
    if (exec > 0.0) {
        q.front = 0.0;
        return exec * lot;
    }
    return 0.0;
}

} // namespace hft
```

8.2 ทดลอง: เล่นซ้ำสถานการณ์เล็ก ๆ และกวาดค่าพารามิเตอร์บน GPU

สถานการณ์: เราวางคำสั่งซื้อ 1 lot ตอนที่ระดับราคามี 10 lot แล้วเกิดเหตุการณ์ 6 ขั้น

1. มีคนต่อท้ายคิว ระดับราคา 10 → 14
2. มีการยกเลิก 4 lot, 14 → 10 (แต่ละแบบจำลองตีความต่างกัน)
3. มีการขายตามราคาตลาด 7 lot ที่ราคาของเรา
4. ปริมาณในสมุดอัปเดตตามการซื้อขายนั้น, 10 → 3
5. มีการขาย 1 lot
6. ปริมาณอัปเดต, 3 → 2

โค้ด `examples/queue_demo.cu` เล่นซ้ำสถานการณ์นี้บน CPU แล้วให้ GPU ทดลองค่า n หลายค่าพร้อมกัน (หนึ่ง thread ต่อหนึ่งค่าพารามิเตอร์) **ผลลัพธ์จากการรันจริง:**

```
step : risk-averse front | prob(n=2) front
   1 : 10.000 | 10.000
   2 : 10.000 | 6.552
   3 : 3.000 | -0.448
   4 : 3.000 | -0.448
   5 : 2.000 | -1.448
   6 : 2.000 | -
risk-averse: filled 0 lot (step 0) | prob(n=2): filled 1 lot (step 5)
```

```
GPU sweep (prob model, f(x)=x^n):
  n= 0.5 front after cancel = 7.550 filled at step 0 (cpu: 7.550, step 0)
  n= 1.0 front after cancel = 7.143 filled at step 5 (cpu: 7.143, step 5)
  n= 2.0 front after cancel = 6.552 filled at step 5 (cpu: 6.552, step 5)
  n= 3.0 front after cancel = 6.241 filled at step 3 (cpu: 6.241, step 3)
  n= 5.0 front after cancel = 6.041 filled at step 3 (cpu: 6.041, step 3)
  n=10.0 front after cancel = 6.000 filled at step 3 (cpu: 6.000, step 3)
```

อธิบายแบบง่าย — อ่านผล

- **ขั้น 4:** ปริมาณลด 7 แต่มีการซื้อขาย 7 อยู่แล้ว ส่วนที่ลดโดยไม่ได้มาจากการซื้อขายจึงเป็น 0 ตำแหน่งไม่ถูกนับซ้ำ นี่คือเหตุผลที่ต้องเก็บปริมาณซื้อขายสะสมไว้
- **Risk-averse** ไม่ได้ซื้อขายเลยในสถานการณ์นี้ (front เหลือ 2) เพราะเชื่อว่าการยกเลิกเกิดข้างหลังเราทั้งหมด
- **Prob model** ได้ซื้อขายเร็วหรือช้าขึ้นกับ n: ค่า n ใหญ่ได้ซื้อขายตั้งแต่ขั้น 3 ส่วน $n = 0.5$ ไม่ได้ซื้อขายเลย
- **ข้อสรุป:** การเลือกแบบจำลองคิวเปลี่ยนผลการทดสอบ market making ได้มาก จึงต้องปรับเทียบ (calibrate) กับผลซื้อขายจริง ตามที่ผู้พัฒนา HftBacktest แนะนำ
- ผลจาก GPU และ CPU ตรงกันทุกค่า เพราะใช้ฟังก์ชันเดียวกัน (`__host__ __device__`)

8.3 การแบ่งงานบน GPU: อะไรทำขนานได้

อธิบายแบบง่าย

การทดสอบย้อนหลังแบบ event-driven ของ**หนึ่ง**กลยุทธ์กับ**หนึ่ง**ชุดข้อมูล เป็นงาน**ตามลำดับเวลา**โดยธรรมชาติ (เหตุการณ์ถัดไปขึ้นกับสถานะก่อนหน้า) จึงไม่เหมาะกับการแบ่งเหตุการณ์ให้หลาย thread ทำพร้อมกัน แต่มีมิติที่ทำขนานได้ดีมาก

มิติที่ทำขนานได้	ตัวอย่าง
ชุดพารามิเตอร์ (parameter sweep)	หนึ่ง thread ต่อค่า $(\gamma, k, \text{spread}, n)$ หนึ่งชุด ทุก thread อ่านเหตุการณ์ชุดเดียวกัน
แบบจำลองคิว/ความหน่วง	เทียบหลายแบบจำลองพร้อมกันกับข้อมูลเดียวกัน
สินทรัพย์หรือวันทำการ	หนึ่ง block ต่อสินทรัพย์หรือต่อวัน (ข้อมูลแยกกัน)
การจำลอง Monte Carlo	หนึ่ง thread ต่อเส้นทางจำลอง (แบบบทความของ NVIDIA)

ข้อดีของ parameter sweep บน GPU: ทุก thread ใน warp อ่านเหตุการณ์ลำดับเดียวกันพร้อมกัน การอ่านหน่วยความจำจึงเป็นแบบ broadcast ที่มีประสิทธิภาพ (ควรเก็บเหตุการณ์ใน global memory แบบอ่านอย่างเดียว) **ข้อควรระวัง:** ถ้า thread ต่าง ๆ แตกแขนง (branch) ต่างกันมาก เช่น บาง thread ได้ซื้อขายบาง thread ไม่ได้ จะเกิด warp divergence และสถานะต่อ thread (คำสั่งที่เปิดอยู่, สมุดคำสั่งซื้อขาย) ต้องเล็กพอจะอยู่ใน register/local memory

8.4 ขั้นตอนที่แนะนำสำหรับโครงงาน

1. อ่านข้อมูลเป็นอาร์เรย์ของ `Event` (64 ไบต์ต่อระเบียน) ตรวจลำดับเวลาและความหน่วงที่ติดลบ
2. เขียน engine บน CPU (C++) ก่อน: สมุดคำสั่งซื้อขาย L2, ความหน่วงแบบค่าคงที่, แบบจำลองคิว risk-averse
3. ตรวจผลกับ HftBacktest (Python) ด้วยข้อมูลและพารามิเตอร์เดียวกัน

4. ย้ายส่วนที่เป็น parameter sweep ไป GPU ด้วย CUDA (และใช้ Thrust สำหรับสรุปผล เช่น หาค่าเฉลี่ย เรียงลำดับ หาค่าที่ดีที่สุด)
5. วัดความเร็วตามหลักในคู่มือ Thrust (อุ่นเครื่อง, `cudaEvent`, ระบุว่ารวมการคัดลอกข้อมูลหรือไม่)

9 ข้อสมมุติ ข้อจำกัด และหลักการตรวจสอบ

สรุปสั้น ๆ

- คำสั่งของเราต้องเล็กจนไม่กระทบตลาด (ข้อมูลที่เล่นซ้ำไม่ตอบสนองต่อเรา)
- ตำแหน่งในคิวจากข้อมูล L2 เป็นการ**ประมาณ** ผลขึ้นกับแบบจำลองที่เลือก
- ความหน่วงต้องวัดจากระบบจริงถ้าเป็นไปได้ ค่าคงที่เป็นเพียงจุดเริ่มต้น
- คำสั่งขนาดใหญ่ที่กินสภาพคล่องให้ผลไม่สมจริง
- **หลักการสำคัญที่สุด:** ทำให้ผลทดสอบย้อนหลังตรงกับผลซื้อขายจริงในอดีตก่อน แล้วจึงปรับแต่งกลยุทธ์

10 อภิธานศัพท์

ภาษาไทย	English	ความหมายสั้น ๆ
การทดสอบย้อนหลัง	backtest	ทดลองกลยุทธ์กับข้อมูลในอดีต
เล่นซ้ำตลาด	market replay	ป้อนเหตุการณ์ในอดีตทีละรายการตามเวลา
ขับเคลื่อนด้วยเหตุการณ์	event-driven	ประมวลผลเหตุการณ์ที่เวลาน้อยที่สุดก่อนเสมอ
ความหน่วงของข้อมูลตลาด	feed latency	local_ts − exch_ts
ความหน่วงขาส่ง / ขาตอบกลับ	entry / response latency	เวลาที่คำสั่งไปถึงตลาด / ผลกลับมาถึงเรา
ตำแหน่งในคิว	queue position	ปริมาณที่อยู่ข้างหน้าคำสั่งของเรา
ลำดับราคา–เวลา	price–time priority	ราคาดีกว่าได้ก่อน ราคาเท่ากันใครมาก่อนได้ก่อน
L2 / MBP	market-by-price	ปริมาณรวมต่อระดับราคา
L3 / MBO	market-by-order	ข้อมูลรายคำสั่ง
ได้ซื้อขายบางส่วน	partial fill	ได้ไม่ครบปริมาณที่สั่ง
post-only (GTX)	post-only	คำสั่งที่ยอมเป็น maker เท่านั้น ถ้าจะจับคู่ทันทีให้ยกเลิก
ค่าธรรมเนียม maker/taker	maker/taker fee	ผู้วางคำสั่งรอ / ผู้กินสภาพคล่อง
เงินคืน	rebate	ค่าธรรมเนียมติดลบที่ตลาดจ่ายให้ maker
ขนาด tick / lot	tick / lot size	ช่วงราคาขั้นต่ำ / ปริมาณขั้นต่ำ
กวาดค่าพารามิเตอร์	parameter sweep	ทดลองหลายชุดพารามิเตอร์พร้อมกัน

แหล่งอ้างอิง

- HftBacktest documentation — `https://hftbacktest.readthedocs.io/en/latest/` (หน้า Getting Started, Data, Latency Models, Order Fill, Probability Queue Models)
- HftBacktest repository (MIT License) — `https://github.com/nkaz001/hftbacktest` (ซอร์สโค้ด แบบ จำลอง คิว: `hftbacktest/src/backtest/models/queue.rs`)

- M. Avellaneda, S. Stoikov, “High-frequency trading in a limit order book,” *Quantitative Finance* 8(3), 2008.
- NVIDIA CCCL documentation (Thrust) — `https://nvidia.github.io/cccl/`